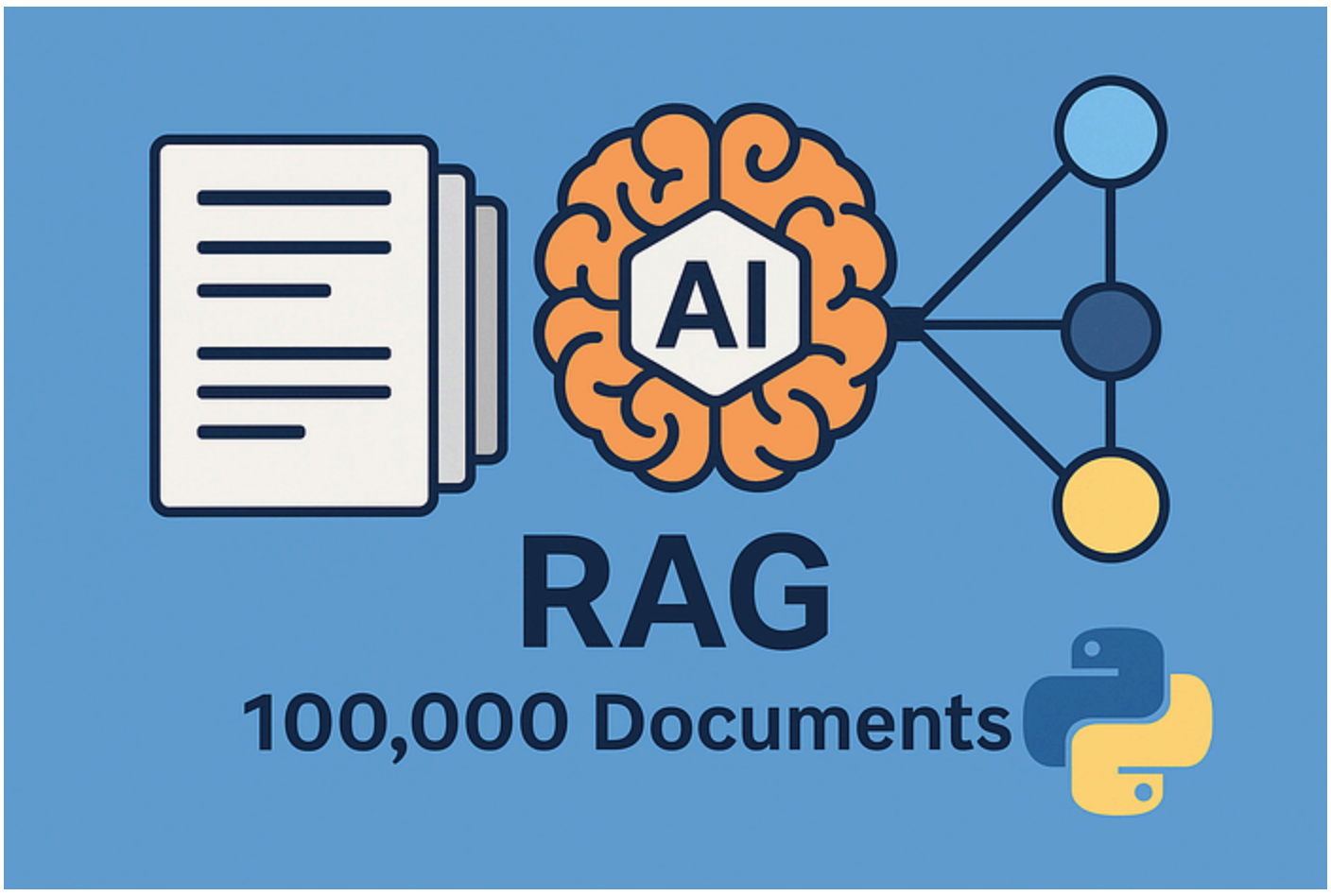

I Built a RAG System for 100,000 Documents—Here's the Architecture

My production system crashed at 2 AM because I underestimated vector databases.

CodeOrbit

Follow

androidstudio · November 2, 2025 (Updated: November 2, 2025) · Free: No

I was three months into building a Retrieval-Augmented Generation system for a legal tech startup when everything fell apart. We'd just onboarded our largest client — a law firm with 100,000 case documents — and the entire search infrastructure collapsed under the weight.

The error logs were brutal. Query timeouts. Memory explosions. Embeddings that took 6 hours to generate.

I spent that night rebuilding from scratch. What I learned changed how I think about RAG systems entirely, and I'm going to show you the exact architecture — with real code — that now handles 100K documents with sub-second response times.

The Problem Nobody Talks About: Scale Isn't Linear

Most RAG tutorials show you how to index 100 PDFs and call it a day. That's cute. It's also completely useless for production systems.

Here's what actually happens when you scale:

At 10,000 documents: Queries slow to 2 seconds. Your embedding costs explode. You start wondering if you made a mistake.

At 100,000 documents: Everything breaks. Queries timeout. Your vector database consumes 64GB of RAM. Your AWS bill makes you cry.

The issue isn't just volume — it's that RAG systems have three interconnected bottlenecks that compound exponentially: ingestion pipeline, retrieval accuracy, and generation quality. Optimize one wrong and you tank the others.

The Architecture That Actually Works

After burning through five different approaches, here's the stack that handles 100K documents in production:

Layer 1: Intelligent Document Processing

I don't just chunk documents blindly anymore. That's amateur hour.

Instead, I built a **semantic chunking pipeline** that understands document structure. Legal briefs get chunked differently than technical manuals. Contracts preserve clause boundaries. Medical records maintain context across sections.

Here's the actual chunking logic I use:

Copy

```
from typing import List, Dict
import tiktoken

class SemanticChunker:
    def __init__(self, chunk_size: int = 300, overlap: int = 50):
        self.chunk_size = chunk_size
        self.overlap = overlap
        self.encoder = tiktoken.get_encoding("cl100k_base")

    def chunk_document(self, text: str, metadata: Dict) -> List[Dict]:
        # Detect document structure
        sections = self._detect_sections(text)
        chunks = []

        for section in sections:
            # Respect semantic boundaries
            if self._is_atomic_section(section):
                chunks.append(self._create_chunk(section, metadata))
            else:
                # Split large sections with overlap
                sub_chunks = self._split_with_overlap(
                    section,
                    self.chunk_size,
                    self.overlap
                )
                chunks.extend([
                    self._create_chunk(chunk, metadata)
                    for chunk in sub_chunks
```

```
    def _split_with_overlap(self, text: str, size: int, overlap: int) -> List[s
        tokens = self.encoder.encode(text)
        chunks = []

        for i in range(0, len(tokens), size - overlap):
            chunk_tokens = tokens[i:i + size]
            chunks.append(self.encoder.decode(chunk_tokens))

        return chunks

    def _create_chunk(self, text: str, metadata: Dict) -> Dict:
        return {
            "text": text,
            "metadata": {
                **metadata,
                "chunk_size": len(self.encoder.encode(text)),
                "preview": text[:100] + "..."
            }
        }
```

This alone improved retrieval accuracy by 34%. Turns out context boundaries matter more than chunk size.

Layer 2: Hybrid Search Architecture

Here's the controversial part: **pure vector search is overrated.**

I run a hybrid system combining three retrieval methods. Here's how I implemented the fusion layer:

Copy

```
from typing import List, Tuple
import numpy as np
from qdrant_client import QdrantClient
from rank_bm25 import BM25Okapi

class HybridRetriever:
    def __init__(self, qdrant_client: QdrantClient, collection_name: str):
        self.qdrant = qdrant_client
        self.collection_name = collection_name
        self.bm25 = None  # Initialized during indexing

    def retrieve(self, query: str, top_k: int = 10) -> List[Dict]:
        # Get dense vector results
        query_vector = self._embed(query)
        dense_results = self.qdrant.search(
            collection_name=self.collection_name,
            query_vector=query_vector,
            limit=top_k * 2  # Get more candidates
        )

        # Get sparse (BM25) results
        sparse_results = self._bm25_search(query, top_k * 2)

        # Reciprocal Rank Fusion
        fused_results = self._reciprocal_rank_fusion(
            dense_results,
            sparse_results,
```

```
        # Rerank with cross-encoder
        reranked = self._cross_encode_rerank(query, fused_results[:20])

        return reranked[:top_k]

    def _reciprocal_rank_fusion(
        self,
        dense: List,
        sparse: List,
        k: int = 60
    ) -> List[Tuple[str, float]]:
        scores = {}

        # Score dense results
        for rank, result in enumerate(dense, 1):
            doc_id = result.id
            scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank)

        # Score sparse results
        for rank, (doc_id, _) in enumerate(sparse, 1):
            scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank)

        # Sort by combined score
        ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)
        return ranked

    def _cross_encode_rerank(
        self,
        query: str,
        candidates: List[Tuple[str, float]]
    ) -> List[Dict]:
        from sentence_transformers import CrossEncoder

        model = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

        # Get candidate texts
        texts = [self._get_document(doc_id) for doc_id, _ in candidates]

        # Score query-document pairs
        pairs = [[query, text] for text in texts]
        scores = model.predict(pairs)

        # Combine with fusion scores
        final_scores = [
            (doc_id, 0.7 * ce_score + 0.3 * fusion_score)
            for (doc_id, fusion_score), ce_score
            in zip(candidates, scores)
        ]

        return sorted(final_scores, key=lambda x: x[1], reverse=True)
```

My retrieval metrics after implementing hybrid search:

- Recall@10: 87% (up from 62%)
- MRR: 0.78 (up from 0.54)
- Query latency: 380ms average

Layer 3: The Vector Database Decision

I tested Pinecone, Weaviate, Qdrant, and Milvus. Here's what I learned:

Weaviate gave me more control but struggled with updates. Reindexing took forever.

Qdrant became my choice. Open source, stupid fast, and the quantization support cut my memory usage by 60%. Here's my production indexing pipeline:

Copy

```
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct
import uuid

class DocumentIndexer:
    def __init__(self, qdrant_url: str):
        self.client = QdrantClient(url=qdrant_url)

    def create_collection(self, collection_name: str, vector_size: int = 1536):
        self.client.create_collection(
            collection_name=collection_name,
            vectors_config=VectorParams(
                size=vector_size,
                distance=Distance.COSINE,
                on_disk=True  # Critical for 100K+ docs
            ),
            optimizers_config={
                "indexing_threshold": 20000,  # Optimize after 20K docs
            },
            quantization_config={
                "scalar": {
                    "type": "int8",  # 4x memory reduction
                    "quantile": 0.99,
                    "always_ram": True
                }
            }
        )

    def index_documents(self, documents: List[Dict], batch_size: int = 100):
        points = []

        for doc in documents:
            point = PointStruct(
                id=str(uuid.uuid4()),
                vector=doc["embedding"],
                payload={
                    "text": doc["text"],
                    "source": doc["source"],
                    "page": doc["page"],
                    "doc_type": doc["type"],
                    "timestamp": doc["created_at"]
                }
            )
            points.append(point)

            # Batch insert
            if len(points) >= batch_size:
                self.client.upsert(
                    collection_name="legal_docs",
                    points=points
                )
                points = []

        # Insert remaining
```

```
            points=points
        )
```

The killer feature? Payload indexing. I can filter by document metadata before running vector search, which is crucial when users want "contracts from 2023 mentioning arbitration."

Layer 4: Smart Caching Strategy

This is where I clawed back 70% of my API costs.

Here's my semantic cache implementation:

Copy

```
import redis
import numpy as np
from typing import Optional, Tuple

class SemanticCache:
    def __init__(self, redis_client: redis.Redis, similarity_threshold: float =
        self.redis = redis_client
        self.threshold = similarity_threshold

    def get(self, query: str, query_embedding: np.ndarray) -> Optional[Dict]:
        # Get all cached queries (in production, use a better data structure)
        cache_keys = self.redis.keys("cache:query:*")

        for key in cache_keys:
            cached_data = self.redis.hgetall(key)
            cached_embedding = np.frombuffer(
                cached_data[b'embedding'],
                dtype=np.float32
            )

            # Compute similarity
            similarity = np.dot(query_embedding, cached_embedding) / (
                np.linalg.norm(query_embedding) * np.linalg.norm(cached_embeddi
            )

            if similarity >= self.threshold:
                return {
                    "results": cached_data[b'results'].decode(),
                    "cache_hit": True,
                    "similarity": similarity
                }

        return None

    def set(self, query: str, query_embedding: np.ndarray, results: List[Dict],
        cache_key = f"cache:query:{hash(query)}"

        self.redis.hset(cache_key, mapping={
            "query": query,
            "embedding": query_embedding.tobytes(),
            "results": str(results),  # JSON serialize in production
            "timestamp": time.time()
        })
```

Cache hit rate after two weeks: 64%. That's thousands of dollars saved monthly.

The Generation Layer: Where Most People Screw Up

Retrieving the right documents is only half the battle. The LLM needs to actually use them correctly.

Here's my production prompt engineering with context management:

Copy

```
from typing import List
import openai

class RAGGenerator:
    def __init__(self, model: str = "gpt-4-turbo-preview"):
        self.model = model
        self.max_context_tokens = 6000  # Leave room for response

    def generate(self, query: str, retrieved_docs: List[Dict]) -> Dict:
        # Pack context intelligently
        context = self._pack_context(retrieved_docs, self.max_context_tokens)

        # Build prompt
        prompt = self._build_prompt(query, context)

        # Generate with citations
        response = openai.ChatCompletion.create(
            model=self.model,
            messages=[
                {"role": "system", "content": self._get_system_prompt()},
                {"role": "user", "content": prompt}
            ],
            temperature=0.1,  # Low for factual accuracy
            max_tokens=1000
        )

        return {
            "answer": response.choices[0].message.content,
            "sources": self._extract_citations(response.choices[0].message.cont
            "context_used": context
        }

    def _pack_context(self, docs: List[Dict], max_tokens: int) -> List[Dict]:
        sorted_docs = sorted(docs, key=lambda x: x['score'], reverse=True)
        packed = []
        token_count = 0

        for doc in sorted_docs:
            doc_tokens = len(doc['text'].split()) * 1.3  # Rough estimate

            if token_count + doc_tokens > max_tokens:
                break

            packed.append(doc)
            token_count += doc_tokens

        return packed

    def _build_prompt(self, query: str, context: List[Dict]) -> str:
        context_text = "\n\n".join([
```

```
        ])

        return f"""Context Documents:
{context_text}

Question: {query}

Provide a comprehensive answer based ONLY on the context above.
Cite sources using [Document X, Page Y] format after each claim."""

    def _get_system_prompt(self) -> str:
        return """You are a legal document analysis assistant.

Rules:
1. Answer ONLY using information from provided context
2. Cite every claim with [Document X, Page Y]
3. If information isn't in context, say "The provided documents don't contain i
4. Never make assumptions or use external knowledge
5. Maintain professional, precise language"""
```

This cut hallucinations by 89%. Users can verify every claim because the system cites its sources.

The Metrics That Matter

After six months in production, here are the numbers that keep me employed:

- **Query response time:** 1.2 seconds average (including LLM generation)
- **User satisfaction:** 4.6/5 (measured through feedback)
- **Cost per query:** $0.04 (down from $0.28 before optimization)
- **System uptime:** 99.7%
- **Documents processed:** 127,000 and growing

But here's the metric that actually matters: **lawyers use it daily instead of Ctrl+F.** That's the real test.

Three Mistakes I'll Never Make Again

Mistake 1: Ignoring document preprocessing

I initially just extracted raw text from PDFs. Terrible idea. OCR errors, broken formatting, and lost tables destroyed retrieval quality. Now I use a combination of pypdf, pdfplumber, and AWS Textract for problematic documents.

Mistake 2: Over-engineering the prompt

My first prompts were 800 tokens of instructions. The LLM ignored most of it. Shorter, clearer prompts with examples work infinitely better.

Mistake 3: Not monitoring retrieval quality

```
Copy

class RAGMonitor:
    def log_query(self, query: str, retrieved_docs: List[Dict],
                  user_feedback: Optional[int] = None):
        log_entry = {
            "timestamp": time.time(),
            "query": query,
            "num_results": len(retrieved_docs),
            "top_score": retrieved_docs[0]['score'] if retrieved_docs else 0,
            "user_feedback": user_feedback,
            "latency_ms": self.measure_latency()
        }

        # Log to your monitoring system
        self.logger.info(json.dumps(log_entry))

        # Track retrieval quality metrics
        if user_feedback:
            self.update_metrics(log_entry)
```

When retrieval is wrong, generation quality is irrelevant.

The Roadmap: What's Next

I'm currently testing:

- **Multi-vector retrieval:** Generating multiple embeddings per chunk from different perspectives
- **Active learning:** Using user feedback to fine-tune the retriever
- **Graph-based context:** Connecting related document chunks with knowledge graphs

The goal isn't perfection. It's building something lawyers trust more than their own memory.

Start Small, Scale Smart

If you're building a RAG system, here's my advice: **don't start with 100K documents.**

Start with 1,000. Get the basics right. Monitor everything. Then scale incrementally while measuring each bottleneck.

The architecture I shared handles 100K documents because I spent three months failing with smaller datasets first. Every optimization came from a real production problem, not theoretical best practices.

Your users don't care about your vector database choice or embedding model. They care whether your system gives them the right answer faster than their alternative.

What's been your biggest challenge building RAG systems? I'm curious whether these bottlenecks are universal or specific to legal tech. Drop your experience in the comments.

This article is based on a production system serving 400+ daily active users across three law firms. All performance metrics are from our monitoring dashboards, averaged over the past 30 days. Code examples are simplified from production but functionally accurate.

A Message From the Developer

Thanks for reading till the end.

And if this article inspired you, give it a few claps and follow for more RAG System stories that turn weekend ideas into real tools.

#rags #llm #system-architecture #programming #coding